CUBE MANUAL

Contents

1 Links 1

2 Introduction 1

3 Basics 2

4 Synchronization 3

5 Facets 4

6 Facet Extension API 5

7 The Default Implementation of `call-with-facet*` 6

8 Lifetime 7
- 8.1 Facet Barriers . . . 8

9 Indices 8
- 9.1 Function and Macro Index . . . 9
- 9.2 Variable and Constant Index . . . 10
- 9.3 Type Index . . . 10
- 9.4 Misc Index . . . 10

[in package MGL-CUBE]

1 Links

The official repository is https://github.com/melisgl/mgl-mat, and this document is available in various formats on https://fixnum.com for the latest version.

2 Introduction

This is the library on which MGL-MAT (see MAT Manual) is built. The idea of automatically translating between various representations may be useful for other applications, so this got its own package and all ties to MGL-MAT has been severed.

This package defines `cube`, an abstract base class that provides a framework for automatic conversion between various representations of the same data. To define a cube, `cube` needs to

be subclassed and the Facet Extension API be implemented.

If you are only interested in how to use cubes in general, read Basics, Lifetime and Facet Barriers.

If you want to implement a new cube datatype, then see Facets, Facet Extension API, and The Default Implementation of `call-with-facet*`.

3 Basics

Here we learn what a `cube` is and how to access the data in it with `with-facet`.

- **[class]** **cube**

  A datacube that has various representations of the same stuff. These representations go by the name 'facet'. Clients must use `with-facet` to acquire a dynamic extent reference to a facet. With the information provided in the `direction` argument of `with-facet`, the cube keeps track of which facets are up-to-date and copies data between them as necessary.

  The cube is an abstract class, it does not provide useful behavior in itself. One must subclass it and implement the Facet Extension API.

  Also see Lifetime and Facet Barriers.

- **[macro]** **with-facet** *(var (cube facet-name &key (direction :io) type)) &body body*

  Find or create the facet with `facet-name` in `cube` and bind `var` to the representation of `cube`'s data provided by that facet. This representation is called the facet's *value*. The value is to be treated as dynamic extent: it is not allowed to keep a reference to it. For the description of the `direction` parameter, see the type `direction`.

  If `type` is specified, then `var` is declared to be of that type.

- **[type]** **direction**

  Used by `with-facet`, `direction` can be `:input`, `:output` or `:io`.

  - `:input` promises that the facet will only be read and never written. Other up-to-date facets of the same cube remain up-to-date. If the facet in question is not up-to-date then data is copied to it from one of the up-to-date facets (see `select-copy-source-for-facet*`).
  - `:output` promises that *all* data will be overwritten without reading any data. All up-to-date facets become non-up-to-date, while this facet is marked as up-to-date. No copying of data takes place.
  - `:io` promises nothing about the type of access. All up-to-date facets become non-up-to-date, while this facet is marked as up-to-date. If the facet in question is not up-to-date then data is copied to it from one of the up-to-date facets (see `select-copy-source-for-facet*`).

  Any number of `with-facet`s with direction `:input` may be active at the same time, but `:io` and `:output` cannot coexists with another `with-facet` regardless of the direction. The exception for this rule is that an inner `with-facet` does not conflict with an enclosing

`with-facet` if they are for the same facet (but inner `with-facet`s for another facet or for the same facet from another thread do).

See `check-no-writers` and `check-no-watchers` called by The Default Implementation of `call-with-facet*`.

- **[macro]** **with-facets** *(&rest facet-binding-specs) &body body*

  A shorthand for writing nested `with-facet` calls.

```
(with-facet (f1 (c1 'name1 :direction :input))
  (with-facet (f2 (c2 'name2 :direction :output))
    ...))
```

  is equivalent to:

```
(with-facets ((f1 (c1 'name1 :direction :input))
              (f2 (c2 'name2 :direction :output)))
  ...)
```

4 Synchronization

Cubes keep track of which facets are used, which are up-to-date to be able to perform automatic translation between facets. `with-facet` and other operations access and make changes to this metadata so thread safety is a concern. In this section, we detail how to relax the default thread safety guarantees.

A related concern is async signal safety which arises most often when C-c'ing or killing a thread or when the extremely nasty WITH-TIMEOUT macro is used. In a nutshell, changes to cube metadata are always made with interrupts disabled so things should be async signal safe.

- **[accessor]** **synchronization** *cube (:synchronization = *default-synchronization*)*

  By default, setup and teardown of facets by `with-facet` is performed in a thread safe way. Corrupting internal data structures of cubes is not fun, but in the name of performance, synchronization can be turned off either dynamically or on a per instance basis.

  If `t`, then access to cube metadata is always synchronized. If `nil`, then never. If `:maybe`, then whether access is synchronized is determined by `*maybe-synchronize-cube*` that's true by default.

  The default is the value of `*default-synchronization*` that's `:maybe` by default.

  Note that the body of a `with-facet` is never synchronized with anyone, apart from the implicit reader/writer conflict (see `direction`).

- **[variable]** ***default-synchronization*** *:maybe*

  The default value for `synchronization` of new cubes.

- **[variable]** ***maybe-synchronize-cube*** *t*

Determines whether access the cube metadata is synchronized for cubes with `synchronization :maybe`.

5 Facets

The basic currency for implementing new cube types is the `facet`. Simply using a cube only involves facet names and values, never facets themselves.

- **[function]** **facets** *cube*

  Return the facets of `cube`.

- **[function]** **find-facet** *cube facet-name*

  Return the facet of `cube` for the facet with `facet-name` or `nil` if no such facet exists.

- **[structure]** **facet**

  A cube has facets, as we discussed in Basics. Facets holds the data in a particular representation, this is called the *value* of the facet. A facet holds one such value and some metadata pertaining to it: its `facet-name`, whether it's up-to-date (`facet-up-to-date-p`), etc. `facet` objects are never seen when simply using a cube, they are for implementing the Facet Extension API.

- **[structure-accessor]** **facet-name** *facet*

  A symbol that uniquely identifies the facet within a cube.

- **[structure-accessor]** **facet-value** *facet*

  This is what's normally exposed by `with-facet`.

- **[structure-accessor]** **facet-description** *facet*

  Returned by `make-facet*` as its second value, this is an arbitrary object in which additional information can be stored.

- **[structure-accessor]** **facet-up-to-date-p** *facet*

  Whether the cube has changed since this facet has been last updated. See `facet-up-to-date-p*`.

- **[structure-accessor]** **facet-n-watchers** *facet*

  The number of active `with-facet`s. Updated by `watch-facet` and `unwatch-facet`.

- **[structure-accessor]** **facet-watcher-threads** *facet*

  The threads (one for each watcher) that have active `with-facet`s.

- **[structure-accessor]** **facet-direction** *facet*

  The direction of the last `with-facet` on this facet.

6 Facet Extension API

Many of the generic functions in this section take `facet` arguments. `facet` is a structure and is not intended to be subclassed. To be able to add specialized methods, the name of the facet (`facet-name`) is also passed as the argument right in front of the corresponding facet argument.

In summary, define `eql` specializers on facet name arguments, and use `facet-description` to associate arbitrary information with facets.

- **[generic-function]** **make-facet*** *cube facet-name*

  Called by `with-facet` (or more directly `watch-facet`) when there is no facet with `facet-name`. As the first value, return a new object capable of storing `cube`'s data in the facet with `facet-name`. As the second value, return a facet description which will be available as `facet-description`. As the third value, return a generalized boolean indicating whether this facet must be explicitly destroyed (in which case a finalizer will be added to `cube`).

- **[generic-function]** **destroy-facet*** *facet-name facet*

  Free the resources associated with `facet` with `facet-name`. The cube this facet belongs to is not among the parameters because this method can be called from a finalizer on the cube (so we can't have a reference to the cube portably) which also means that it may run in an unpredictable thread.

- **[generic-function]** **copy-facet*** *cube from-facet-name from-facet to-facet-name to-facet*

  Copy the `cube`'s data from `from-facet` with `from-facet-name` to `to-facet` with `to-facet-name`. Called by `with-facet` (or more directly `watch-facet`) when necessary. `from-facet` is what `select-copy-source-for-facet*` returned.

- **[generic-function]** **call-with-facet*** *cube facet-name direction fn*

  Call `fn` with an up-to-date `facet-value` that belongs to `facet-name` of `cube`. `with-facet` is directly implemented in terms of this function. See The Default Implementation of `call-with-facet*` for the gory details.

  Specializations will most likely want to call the default implementation (with `call-next-method`) but with a lambda that transforms `facet-value` before passing it on to `fn`.

- **[generic-function]** **facet-up-to-date-p*** *cube facet-name facet*

  Check if `facet` with `facet-name` has been updated since the latest change to `cube` (that is, since the access to other facets with `direction` of `:io` or `:output`). The default method simply calls `facet-up-to-date-p` on `facet`.

  One reason to specialize this is when some facets actually share common storage, so updating one make the other up-to-date as well.

- **[generic-function]** **select-copy-source-for-facet*** *cube to-name to-facet*

  Called when `to-facet` with `to-name` is about to be updated by copying data from an up-to-date facet. Return the facet (or its name) from which data shall be copied. Note that if the returned facet is not `facet-up-to-date-p`, *then it will be updated first and another*

SELECT-COPY-SOURCE-FOR-FACET will take place, so be careful not to get into endless recursion. The default method simply returns the first up-to-date facet.

PAX integration follows, don't worry about it if you don't use PAX, but you really should (see PAX Manual).

- **[locative]** **facet-name**

  The `facet-name` locative is to refer to stuff defined with `define-facet-name`.

- **[macro]** **define-facet-name** *symbol lambda-list &body docstring*

  Just a macro to document that `symbol` refers to a facet name (as in the `facet-name`). This is totally confusing, so here is an example of how MGL-MAT (see MAT Manual) documents the `mgl-mat:backing-array` facet:

```
(define-facet-name backing-array ()
  "The corresponding facet is a one dimensional lisp array.")
```

  Which makes it possible to refer to this definition (refer as in link and `m-.` to) `mgl-mat:backing-array` facet-name. See PAX Manual for more.

Also see The Default Implementation of `call-with-facet*`.

7 The Default Implementation of `call-with-facet*`

- **[method]** **call-with-facet*** *(cube cube) facet-name direction fn*

  The default implementation of `call-with-facet*` is defined in terms of the `watch-facet` and the `unwatch-facet` generic functions. These can be considered part of the Facet Extension API.

- **[generic-function]** **watch-facet** *cube facet-name direction*

  This is what the default `call-with-facet*` method, in terms of which `with-facet` is implemented, calls first. The default method takes care of creating facets, copying and tracking up-to-dateness.

  Calls `check-no-writers` (unless `*let-input-through-p*`) and `check-no-watchers` (unless `*let-output-through-p*`) depending on `direction` to detect situations with a writer being concurrent to readers/writers because that would screw up the tracking of up-to-dateness.

  The default implementation should suffice most of the time. MGL-MAT specializes it to override the `direction` arg, if it's `:output` but not all elements are visible due to reshaping, so that invisible elements are still copied over.

- **[generic-function]** **unwatch-facet** *cube facet-name*

  This is what the default `call-with-facet*` method, in terms of which `with-facet` is implemented, calls last. The default method takes care of taking down facets. External resource managers may want to hook into this to handle unused facets.

- **[variable]** ***let-input-through-p*** *nil*

  If true, `with-facets` (more precisely, the default implementation of `call-with-facet*`) with `:direction :input` does not call `check-no-writers`. This knob is intended to be bound locally for debugging purposes.

- **[variable]** ***let-output-through-p*** *nil*

  If true, `with-facets` (more precisely, the default implementation of `call-with-facet*`) with `:direction :io` or `:output` does not call `check-no-watchers`. This knob is intended to be bound locally for debugging purposes.

- **[function]** **check-no-writers** *cube facet-name message-format &rest message-args*

  Signal an error if `cube` has facets (with names other than `facet-name`) being written (i.e. direction is `:io` or `:output`).

- **[function]** **check-no-watchers** *cube facet-name message-format &rest message-args*

  Signal an error if `cube` has facets (with names other than `facet-name`) being regardless of the direction.

8 Lifetime

Lifetime management of facets is manual (but facets of garbage cubes are freed automatically by a finalizer, see `make-facet*`). One may destroy a single facet or all facets of a cube with `destroy-facet` and `destroy-cube`, respectively. Also see Facet Barriers.

- **[function]** **destroy-facet** *cube facet-name*

  Free resources associated with the facet with `facet-name` and remove it from `facets` of `cube`.

- **[function]** **destroy-cube** *cube*

  Destroy all facets of `cube` with `destroy-facet`.

In some cases it is useful to declare the intent to use a facet in the future to prevent its destruction. Hence, every facet has reference count which starts from 0. The reference count is incremented and decremented by `add-facet-reference-by-name` and `remove-facet-reference-by-name`, respectively. If it is positive, then the facet will not be destroyed by explicit `destroy-facet` and `destroy-cube` calls, but it will still be destroyed by the finalizer to prevent resource leaks caused by stray references.

- **[function]** **add-facet-reference-by-name** *cube facet-name*

  Make sure `facet-name` exists on `cube` and increment its reference count. Return the `facet` behind `facet-name`.

- **[function]** **remove-facet-reference-by-name** *cube facet-name*

  Decrement the reference count of the facet with `facet-name` of `cube`. It is an error if the facet does not exists or if the reference count becomes negative.

- **[function]** **remove-facet-reference** *facet*

  Decrement the reference count of `facet`. It is an error if the facet is already destroyed or if the reference count becomes negative. This function has the same purpose as `remove-facet-reference-by-name`, but by having a single `facet` argument, it's more suited for use in finalizers because it does not keep the whole `cube` alive.

8.1 Facet Barriers

A facility to control lifetime of facets tied to a dynamic extent. Also see Lifetime.

- **[macro]** **with-facet-barrier** *(cube-type ensures destroys) &body body*

  When `body` exits, destroy facets which:

  - are of cubes with `cube-type`
  - have a facet name among `destroys`
  - were created in the dynamic extent of `body`

  Before destroying the facets, it is ensured that facets with names among `ensures` are up-to-date. `with-facet-barrier`s can be nested, in case of multiple barriers matching the cube's type and the created facet's name, the innermost one takes precedence.

  The purpose of this macro is twofold. First, it makes it easy to temporarily work with a certain facet of many cubes without leaving newly created facets around. Second, it can be used to make sure that facets whose extent is tied to some dynamic boundary (such as the thread in which they were created) are destroyed.

- **[function]** **count-barred-facets** *facet-name &key (type 'cube)*

  Count facets with `facet-name` of cubes of `type` which will be destroyed by a facet barrier.

9 Indices

Referrer definition type abbreviations:

- *f*: for definitions in the function namespace (macros, compiler macros and also methods)
- *t*: `DEFTYPE`s, classes, conditions, structs
- *d*: documentation sections and glossary terms
- *l*: definitions of definition types
- *s*: ASDF systems
- *p*: packages
- *n*: named readtables
- *v*: special variables and constants
- *r*: restarts

- *?*: other

9.1 Function and Macro Index

`add-facet-reference-by-name` 7 *(fn)* ↩ *d*: Lifetime 7
`call-with-facet*` 5 *(gf)*
    ↩ *f*: `unwatch-facet` 6, `watch-facet` 6
    ↩ *v*: `*let-input-through-p*` 7, `*let-output-through-p*` 7
`check-no-watchers` 7 *(fn)*
    ↩ *f*: `watch-facet` 6
    ↩ *t*: `direction` 2
    ↩ *v*: `*let-output-through-p*` 7
`check-no-writers` 7 *(fn)*
    ↩ *f*: `watch-facet` 6
    ↩ *t*: `direction` 2
    ↩ *v*: `*let-input-through-p*` 7
`copy-facet*` 5 *(gf)*
`count-barred-facets` 8 *(fn)*
`define-facet-name` 6 *(macro)* ↩ *l*: `facet-name` 6
`destroy-cube` 7 *(fn)* ↩ *d*: Lifetime 7
`destroy-facet` 7 *(fn)*
    ↩ *d*: Lifetime 7
    ↩ *f*: `destroy-cube` 7
`destroy-facet*` 5 *(gf)*
`facet-description` 4 *(structure-accessor facet)*
    ↩ *d*: Facet Extension API 5
    ↩ *f*: `make-facet*` 5
`facet-direction` 4 *(structure-accessor facet)*
`facet-n-watchers` 4 *(structure-accessor facet)*
`facet-name` 4 *(structure-accessor facet)*
    ↩ *d*: Facet Extension API 5
    ↩ *t*: `facet` 4
`facet-up-to-date-p` 4 *(structure-accessor facet)*
    ↩ *f*: `facet-up-to-date-p*` 5, `select-copy-source-for-facet*` 5
    ↩ *t*: `facet` 4
`facet-up-to-date-p*` 5 *(gf)* ↩ *f*: `facet-up-to-date-p` 4
`facet-value` 4 *(structure-accessor facet)* ↩ *f*: `call-with-facet*` 5
`facet-watcher-threads` 4 *(structure-accessor facet)*
`facets` 4 *(fn)* ↩ *f*: `destroy-facet` 7
`find-facet` 4 *(fn)*
`make-facet*` 5 *(gf)*
    ↩ *d*: Lifetime 7
    ↩ *f*: `facet-description` 4
`remove-facet-reference` 8 *(fn)*
`remove-facet-reference-by-name` 7 *(fn)*
    ↩ *d*: Lifetime 7
    ↩ *f*: `remove-facet-reference` 8
`select-copy-source-for-facet*` 5 *(gf)*
    ↩ *f*: `copy-facet*` 5
    ↩ *t*: `direction` 2
`unwatch-facet` 6 *(gf)* ↩ *f*: `call-with-facet*` 6, `facet-n-watchers` 4
`watch-facet` 6 *(gf)* ↩ *f*: `call-with-facet*` 6, `copy-facet*` 5, `facet-n-watchers` 4, `make-facet*` 5
`with-facet` 2 *(macro)*

↩ *d*: Basics 2, Synchronization 3
↩ *f*: call-with-facet* 5, copy-facet* 5, facet-direction 4, facet-n-watchers 4, facet-value 4, facet-watcher-threads 4, make-facet* 5, synchronization 3, unwatch-facet 6, watch-facet 6, with-facets 3
↩ *t*: cube 2, direction 2
with-facet-barrier 8 *(macro)*
with-facets 3 *(macro)* ↩ *v*: *let-input-through-p* 7, *let-output-through-p* 7

9.2 Variable and Constant Index

default-synchronization 3 *(var)* ↩ *f*: synchronization 3
let-input-through-p 7 *(var)* ↩ *f*: watch-facet 6
let-output-through-p 7 *(var)* ↩ *f*: watch-facet 6
maybe-synchronize-cube 3 *(var)* ↩ *f*: synchronization 3

9.3 Type Index

cube 2 *(class)*
↩ *d*: Basics 2, Introduction 1
↩ *f*: call-with-facet* 6, count-barred-facets 8, remove-facet-reference 8
direction 2 *(type)* ↩ *f*: facet-up-to-date-p* 5, synchronization 3, with-facet 2
facet 4 *(structure)*
↩ *d*: Facet Extension API 5, Facets 4
↩ *f*: add-facet-reference-by-name 7

9.4 Misc Index

call-with-facet* 6 *(method (cube t t t))*
facet-name 6 *(locative)* ↩ *f*: define-facet-name 6
synchronization 3 *(accessor cube)* ↩ *v*: *default-synchronization* 3, *maybe-synchronize-cube* 3